Computers & Security 172 (2027) 105137

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Hardware-anchored assurance: A trusted resilience framework for critical edge infrastructure

Xinzhu Jiang, Bo Zhao *, Peiwen Chen, Chunyu Yang, Juncheng Tong

School of Cyber Science and Engineering, Wuhan University, Bayi Street, Wuhan, 430000, Hubei, China

ARTICLE INFO

Keywords:
Advanced persistent threat (APT)
Cloud services
Cyber resilience model
Security architecture

ABSTRACT

As cloud infrastructure expands into the Cloud-Edge Continuum, edge security hinges on the integrity of critical artifacts (e.g., firmware), which serve as primary targets for persistent threats. Existing hardware-based measures (e.g., ROM) lack flexibility for dynamic workloads, while software agents suffer from expansive Trusted Computing Bases (TCB) and vulnerability to privilege escalation. However, simplistic integration often fails to establish rigorous security boundaries, failing to bridge the contextual disparity between static hardware guarantees and dynamic software vulnerabilities.

Motivated by these challenges, our work presents a hardware-anchored Resilience Capability Framework designed to clearly delineate edge security boundaries. Rather than proposing entirely new hardware primitives, we introduce a Dual-Lock mechanism that systematically integrates established hardware trust anchors with software-defined policies. This mechanism enables Adaptive Critical File Protection, allowing dynamic designation of immutable targets without hardware storage constraints. By empowering software with trusted hardware capabilities, we effectively minimize the attack surface while preserving artifact integrity. Importantly, the framework enforces a Resilient Recovery mechanism capable of executing secure restoration even under non-cooperative conditions.Validated across both a resource-constrained ARM platform (Raspberry Pi) and a production-grade x86 industrial platform, experimental results demonstrate successful recovery from compromised states with manageable overhead. This cross-architecture evaluation provides evidence of portability and scalability across the evaluated architectures within the cloud–edge ecosystem.

1. Introduction

Rapid global expansion toward the network edge demands urgent edge infrastructure hardening as vulnerabilities increase (Geetanshi et al., 2025; Sheikh et al., 2025). Edge computing's adoption of cloud-native virtualization and distributed architectures enables scalable operations but inherently expands attack surfaces, multiplying security risks. Specifically, edge risks stem from inherited shared-isolation conflicts: Virtualization's pooled hardware enables container escapes (Mahavaishnavi et al., 2025)/side-channel attacks (Pandey et al., 2025); multi-tenant flaws permit lateral movement/data breaches (Alobaywi et al., 2026). Notably, the ultimate goal of these diverse attack vectors is often tamper with system integrity, specifically targeting critical files to establish persistence or escalate privileges. To address these threats, and in alignment with the NIST definition of resilience (Ross et al., 2021) requiring integrated detection, protection, and recovery, it is necessary to establish effective resilience protection mechanisms for critical files, thereby safeguarding the information security of the cloud–edge ecosystem under complex attack scenarios.

The resilience of critical files in edge infrastructure is essential. Firstly, critical system components including firmware modules, security configuration files, and core application binaries constitute fundamental components of edge nodes, directly governing their operational stability and security assurance (Ul Haq et al., 2023). The integrity of these files constitutes a critical determinant in maintaining edge service reliability and sustaining user confidence. Secondly, the ability to recover critical files is paramount in the face of increasingly sophisticated cyberattacks and unforeseen system failures (Ta et al., 2025). Should these vital files become corrupted, encrypted by ransomware, or otherwise compromised, the absence of robust recovery capabilities in distributed edge environments can lead to prolonged service disruptions, significant financial losses, and irreparable damage to an organization's reputation.

* Corresponding author.
E-mail addresses: jiangxinzhu@whu.edu.cn (X. Jiang), Zhaobo@whu.edu.cn (B. Zhao), mjzjzc@whu.edu.cn (P. Chen), yangchunyu@whu.edu.cn (C. Yang), jctong@whu.edu.cn (J. Tong).

https://doi.org/10.1016/j.cose.2026.105137
Received 18 March 2026; Received in revised form 19 August 2026; Accepted 30 August 2026
Available online 6 September 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Currently, the protection of critical files within edge infrastructure is predominantly approached via two main categories of implementation. The first category encompasses hardware-based static protection mechanisms, leveraging technologies such as embedded Hardware Security Modules (HSMs) and Trusted Platform Modules (TPMs) (Latif et al., 2025; Aaraj et al., 2009). These methods primarily employ cryptographic techniques to ensure file confidentiality and hash functions to verify file integrity. Additionally, read-only hardware, like ROM, can be used to enforce access control for these files (Rother and Chen, 2024). The second category consists of software-based critical file protection mechanisms. Examples include host-based intrusion detection systems (HIDS) and File Integrity Monitoring (FIM) tools that validate files against known baselines (Efe and Abacı, 2022). File protection is also commonly achieved through storage-level snapshotting and Backup and Disaster Recovery (BDR) platforms designed to restore data following corruption events.

However, hardware-centric approaches inherently suffer from rigidity, rendering them ill-suited to the frequent configuration updates and dynamic application lifecycles characteristic of critical files in edge environments. Moreover, while encryption and integrity verification offer certain safeguards, they do not intrinsically prevent the runtime tampering or destruction of critical files (Peddoju et al., 2020). Conversely, while software-based methods afford superior flexibility, they struggle to maintain their own security integrity when confronted with sophisticated attack scenarios prevalent in edge infrastructure, such as Advanced Persistent Threats (APTs) and zero-day exploits (Wang et al., 2024). Since these mechanisms operate within the same execution context as the operating system, they inherently rely on system-level cooperation to function. As a result, once an adversary gains elevated privileges, these software solutions are susceptible to compromise, bypass, or interception, rendering them ineffective in a non-cooperative scenario (Yang et al., 2026).

Ultimately, the aforementioned traditional methods fail to achieve comprehensive cyber resilience. The prevailing practice of simplistically layering these technologies fails to bridge the contextual disparity between static hardware guarantees and dynamic software vulnerabilities (Jain et al., 2014). This misalignment results in ambiguous security boundaries, where trusted hardware resources remain isolated from the execution context they are meant to protect, preventing effective cross-layer synergy. To address this gap, we propose TRCI (Trusted Resilience for Critical Infrastructure), a hardware-anchored resilience and recovery framework specifically designed for edge environments. Rather than introducing entirely new conceptual primitives, TRCI contributes a systematic integration of established hardware trust anchors (e.g., TPMs and hardware watchdogs) with privileged software monitors. By implementing a Dual-Lock mechanism, our approach unifies hardware and software defenses to establish a robust protection lifecycle for critical files.

Referencing the NIST Cyber Resiliency Framework, TRCI initiates from trusted computing hardware as the root of trust, propagating this trust through a hardware-software co-design approach to establish comprehensive resilience for critical files in edge infrastructure. It enables administrators to dynamically extend protection coverage along with system event-triggered proactive detection, ensuring integrity in complex edge environments, while providing autonomous trusted state recovery even in non-cooperative device scenarios, thereby enhancing both the security and availability of edge infrastructure.

The major contributions are as follows.

(1) This paper proposes a Resilience Capability Framework tailored for cloud–edge infrastructure. By systematically integrating established hardware-assisted resilience concepts, this framework implements the NIST Cyber Resiliency architecture in distributed environments. Bridging the contextual disparity between static hardware guarantees and dynamic software states, this framework rigorously delineates security boundaries for edge nodes. It specifically targets the protection of critical system artifacts, addressing the fundamental requirement for integrating end-to-end resilience capabilities into the cloud–edge continuum.
(2) This paper presents a Dual-Lock mechanism that achieves a cross-layer orchestration of established hardware Roots of Trust and software-defined policies. To overcome the 'Non-Cooperative State' where recovery attempts are actively suppressed within the compromised execution context, we introduce a Resilient Recovery mechanism. This approach secures the control plane against malicious attacks, guaranteeing the atomic restoration of a trusted state even when the host system is under adversarial control.
(3) This paper implements a full prototype to validate the framework's efficacy against advanced persistent threats. Through comprehensive evaluations on both a representative ARM-based edge node and a production-grade x86 industrial platform, the results demonstrate that the system effectively detects and mitigates both Data Plane integrity violations (Section 3.4.1) and Control Plane subversion attempts (Section 3.4.2). The cross-architecture evaluation confirms that our solution achieves these robust security guarantees with manageable overhead, proving its operational scalability from resource-constrained IoT devices to high-performance edge infrastructure.

The remainder of this paper is organized as follows. Section 2 reviews related work in cyber resilience and identifies the limitations of existing hardware and software approaches. Section 3 defines the threat model, system entities, and security boundaries. Section 4 details the design of the proposed Resilience Capability Framework, including the Dual-Lock Architecture and the autonomous recovery mechanism. Section 5 provides a comprehensive security analysis against the defined attack surfaces. Section 6 presents the framework evaluation across both resource-constrained ARM and production-grade x86 platforms. Section 7 delineates the operational limitations and physical security boundaries of our approach. Finally, Section 8 concludes the paper.

2. Related work

2.1. Cyber resilience

The field of cyber resilience has emerged as a critical evolution from traditional, prevention-centric cybersecurity. Academic and industry research defines resilience as the ability of a system to anticipate, withstand, recover from, and adapt to adverse conditions, disruptions, or attacks (Alhidaifi et al., 2024; Hubbard, 2023; Christine and Thinyane, 2022). This paradigm acknowledges the high probability of and shifts focus toward maintaining mission-critical functions during an attack and restoring services to a trusted state afterward. Foundational guidance is provided by the National Institute of Standards and Technology (NIST) (Bartock et al., 2018). While the NIST Cyber Resiliency Framework outlined in SP 800–160 offers a high-level engineering methodology for building resilient systems, NIST Special Publication 800–193, "Platform Firmware Resiliency (PFR) Guidelines", provides specific, actionable technical principles. PFR codifies resilience into three core functions: Protect, to maintain firmware integrity; Detect, to identify unauthorized modifications; and Recover, to restore firmware to a state of integrity automatically. The application of these principles is paramount for the protection of distributed edge infrastructure. The integrity of the underlying edge node, from the firmware up, serves as the root of trust; if this foundation is compromised, any software-based security controls intended to protect critical system artifacts can be inherently undermined (Mahavaishnavi et al., 2025; Marchand et al., 2023).

2.2. Hardware-based resilience practices

To implement robust cyber resilience, hardware-based trust anchors are widely adopted to establish an immutable foundation for system recovery. At the foundational level, Read-Only Memory (ROM) serves as the Core Root of Trust for Measurement (CRTM), ensuring the integrity of the initial boot sequence (Al-Otaiby et al., 2026). Building upon this, Discrete Trusted Platform Modules (TPMs) provide essential services for distributed edge nodes, including platform integrity measurement, data sealing, and remote attestation to verify device health (Ott et al., 2023; Upadhyay et al., 2025; Arthur and Challener, 2015). Complementarily, Secure Elements (SE) and Hardware Security Modules (HSMs) function as tamper-resistant vaults (Garb et al., 2021), specifically dedicated to safeguarding the cryptographic keys necessary for authenticating and decrypting recovery images. Additionally, Trusted Execution Environments (TEEs), such as ARM TrustZone, are employed to isolate critical recovery agents from the untrusted host OS (Gupta, 2023; Pettersen et al., 2017). However, relying solely on these mechanisms presents significant limitations in dynamic edge environments. Static anchors like ROM enforce absolute immutability, which fundamentally clashes with the frequent firmware updates and dynamic application lifecycles required by modern edge workloads (Huang and Wang, 2025). In addition, discrete components like TPMs and SEs are constrained by limited internal storage; they cannot house bulky system artifacts, forcing these files to remain on mutable external storage. Most critically, their protective model is primarily passive: while effective at detecting integrity violations via measurement, they lack the capability to actively prevent runtime destruction—such as malicious deletion or renaming—of critical artifacts stored on the host filesystem.

2.3. Software-based resilience practices

The majority of current resilience and recovery solutions are implemented in software. These include traditional backup and disaster recovery (BDR) platforms, local snapshotting mechanisms, and log-based auditing through host-based agents (Tamimi et al., 2019; Syed, 2024). Software-based File Integrity Monitoring (FIM) tools are also widely used to detect unauthorized changes to critical files by comparing them against a known-good baseline, with the Tripwire system being a seminal example (Liu and Jiang, 2023; Martins et al., 2022). The primary limitation of these methods is their dependency on the integrity of the underlying operating system. Lacking protection from a hardware root of trust, these software tools present a large attack surface and suffer from an expansive Trusted Computing Base (TCB). Lacking a guaranteed secure environment to safeguard their operations, these tools face inherent structural vulnerabilities. An advanced attacker with privileged access could potentially circumvent protection agents or manipulate the underlying storage visibility to invalidate recovery efforts (Eisoldt et al., 2025). Therefore, these software-only solutions are fundamentally incapable of executing a trusted recovery in a non-cooperative state—that is, when the system they are running on has been compromised and actively resists remediation. This inability to guarantee the integrity of the recovery process itself represents a significant security gap, failing to bridge the contextual disparity between static hardware trust and dynamic software states (Dharsee, 2023; Dharsee and Criswell, 2023).

2.4. Hardware-assisted detection and cross-layer resilience

To bridge the semantic gap between static hardware anchors and vulnerable software environments, recent literature emphasizes cross-layer integration. A recent taxonomy structures this design space, categorizing approaches from software-managed hardware detection to embedded cybersecure-by-design architectures (Marchetti et al., 2026).

A prominent research vector utilizes micro-architectural telemetry, such as Hardware Performance Counters (HPCs), often coupled with machine learning, to dynamically identify software violations (Foreman, 2018; Chenet et al., 2024). These approaches analyze runtime execution behavior to detect stealthy threats like fileless malware without relying on static signatures, providing a crucial security layer for resource-constrained IoT environments (Rahman et al., 2017). However, while HPC-based monitors deliver high-fidelity diagnostics, they are inherently detection-centric. Because their remediation logic typically relies on the host operating system, a sophisticated adversary that successfully compromises the kernel could potentially suppress these response mechanisms. In such non-cooperative scenarios, relying solely on software-triggered remediation highlights the need for an independent, hardware-enforced recovery path.

To address the necessity of recovery, cybersecure-by-design paradigms integrate fault-tolerance directly into the hardware logic. Solutions include hardware-enforced Control Flow Integrity (CFI) implemented via custom FPGA soft-cores, utilizing sacrificial co-processors to manage localized thread rollback and tamper evidence (DaSilva and Fortier, 2019). Although these customized System-on-Chip (SoC) architectures provide robust, isolated execution recovery, their reliance on modified instruction pipelines restricts portability across commercial-off-the-shelf (COTS) edge infrastructure. Furthermore, their primary focus remains on instruction-level control flow rather than persistent access control over the file system.

3. Threat model

This methodology is primarily designed for the protection of critical infrastructure within distributed edge environments. To clearly delineate security boundaries and facilitate a robust security analysis, this work defines the following threat model.

3.1. System entities

The architecture's threat model is constructed around three core entities, drawing concepts from NIST and TCG frameworks (Bartock et al., 2018):

- **Resilience Target (RT):** The RT represents the mutable operational environment (e.g., Host OS, firmware, and business applications). It is responsible for the system's primary **Data Plane** functions. In our model, the RT represents a mutable attack surface where compromise could potentially lead to unauthorized kernel-level (Ring 0) access to standard system resources.
- **Resilience Engine (RE):** The RE is the trusted agent co-located within the host environment. It acts as the local enforcement point for the **Control Plane**. While it operates within the same kernel space as the RT to bridge the semantic gap, its root of trust is anchored in immutable hardware (TPM and Watchdog), not the OS.
- **Resilience Authority (RA):** The RA is the remote, trusted decision center. It manages the policy and authorization for the Control Plane. We assume the RA itself is secure; however, the communication channel is exposed to network threats.

3.2. Security boundaries

We classify the interactions into two distinct planes, defining the attack surfaces:

B1: The Data Plane Boundary (B1: Local Resilience). This boundary lies between the RE and the RT on the local device. Since the RT processes untrusted external data and may contain vulnerabilities, this boundary is subject to privilege escalation attacks. The RE must protect the Security Baseline from the RT's interference.

B2: The Control Plane Boundary (B2: Secure Signaling). This boundary represents the command-and-control channel between the local RE and the remote RA. This channel carries critical signaling (e.g., Heartbeats, Reset Directives) and is subject to network-level adversarial actions.

3.3. Design assumptions and scope

To address the inherent complexities of distributed systems, we make the following explicit assumptions. These assumptions contextualize the capabilities of our proposed mechanisms:

We assume the external network infrastructure constituting the Control Plane is reliable and available. The threat model regarding communication interruption is explicitly restricted to the **Resilience Target (RT)** acting as a malicious endpoint. External Network Denial-of-Service (DoS) attacks or upstream infrastructure failures are considered out of the scope of this work. As such, within our scope, any interruption in Control Plane signaling is attributed to a non-cooperative RT.

We assume the underlying hardware (TPM 2.0 and Hardware Watchdog) are trusted. The adversary may control the OS kernel and Host filesystem, but cannot physically tamper with the TPM chip or the dedicated hardware timer layout.

3.4. Adversarial capabilities and attack vectors

Based on the boundaries and assumptions above, we define the following specific attack vectors.

3.4.1. Attacks on the data plane (Boundary B1)

These vectors correspond to an adversary attempting to subvert the host system.

- **A1: Runtime File Tampering.** The adversary uses standard system calls to modify critical files (e.g., `/etc/passwd`, firmware) while the OS is running.
- **A2: Load-Time Backdoor Implantation.** The adversary implants static backdoors or replaces binaries on the disk, intended to execute upon the next reboot or application launch.
- **A3: In-Memory/Fileless Attack.** The adversary executes malicious code directly in memory without modifying files, bypassing static integrity checks.
- **A4: Refusal of Execution (Non-Cooperative State).** This represents the "Worst-Case Scenario" where the adversary has gained Ring 0 control or induced a fatal system crash via memory corruption (e.g., local denial-of-service exploits on commercial edge routers (China National Vulnerability Database (CNVD), 2026)). The RT actively resists remediation by masking interrupts or halting execution, effectively disabling software-based monitoring (Lock 1) or spoofing status reports. This necessitates the hardware-triggered Lock 2.

3.4.2. Attacks on the control plane (Boundary B2)

These vectors target the signaling between RE and RA.

- **N1: Command Forgery.** An adversary attempts to inject fabricated "Reset" or "Update" directives to trigger unauthorized actions.
- **N2: Command Replay.** An adversary captures a valid, historic "Deferral Ticket" and replays it to prevent a necessary reset, attempting to keep a compromised system alive.
- **N3: Control Channel Denial-of-Service.** The compromised RT drops legitimate commands or heartbeats from the RA, attempting to isolate the RE. Under our "Security over Availability" assumption, this leads to a forced reset.

4. System design

4.1. Overview

Within the overall architecture for edge device resilience as shown in Fig. 1, the primary protective subject is the Resilience Target (RT), which encompasses critical assets such as firmware and business applications operating on the device. The on-device enforcement is handled by a Resilience Engine (RE) comprised of two main components.

The core of the RE is the Secure Monitor, which operates at a privileged system level (e.g., Kernel Mode, corresponding to Ring 0 on x86 or EL1 on ARM) to perform continuous integrity monitoring of the RT. The monitor reports the state of the RT to a central Resilience Authority (RA) in real-time. Concurrently, it receives security policies from the RA to execute corresponding remediation actions, such as restoration or updates, on the RT.

The second component is trusted hardware, exemplified by a Trusted Platform Module (TPM). The TPM provides the RE with hardware-anchored cryptographic operations and key management services. Critically, cryptographic keys are perpetually confined within the TPM's security boundary during use, thereby preventing their compromise by malicious software. This design ensures the security of keys Used throughout the processes of integrity measurement, detection, and attestation signaling.

Finally, the Resilience Authority (RA) functions as the central policy decision point. It receives status information from the edge device, evaluates it against pre-configured administrator policies to initiate a corresponding event response, and dispatches the resultant security policy to the device's RE for enforcement.

4.2. Lock1: Security baseline protection mechanism

In the context of security restoration for edge infrastructure, the Security Baseline File (SBF) comprises two fundamental components: a golden measurement for the Resilience Target (RT) and a trusted recovery image. The former serves as a cryptographic benchmark for validating the integrity of the RT, while the latter is the artifact used to restore the RT to a known-good state through remediation.

Given that the Resilience Engine (RE) and the Resilience Target (RT) are co-located on the same physical hardware, a compromised RT could potentially tamper with the security baseline utilized by the RE itself. Such an attack would fundamentally undermine the device's entire resilience and remediation capability. Conventional approaches for protecting this security baseline often rely on read-only hardware. However, such static methods are inherently inflexible. Protected assets are not immutable; firmware requires periodic updates and patching, which necessitates corresponding updates to its golden measurement. Beyond this, the scope of protected content may evolve throughout the device's lifecycle, for instance, with the deployment of new business applications. A read-only hardware approach precludes this critical lifecycle management. Therefore, it is imperative to establish a flexible and extensible protection mechanism that allows protection to be programmatically enabled and disabled.

To address this challenge, we introduce **Lock 1: Security Baseline Protection Mechanism**. At the core of this mechanism lies the Non-Volatile Memory Protection Module (NVPM) within the Secure Monitor, which is specifically designed to ensure the security baseline remains uncompromised. The Resilience Engine leverages the NVPM to mediate access requests based on the security context of the initiating process and the custom protection policy of the SBF. Consequently, when a malicious process outside the RE (i.e., the RT) attempts to modify the SBF, the NVPM intercepts the underlying system call, thereby enforcing the access control policy and preserving the integrity of the protected asset. **Lock 1** effectively encapsulates this logic to enable flexible control over the SBF. The detailed execution flow of this protection mechanism is presented in Algorithm 1.

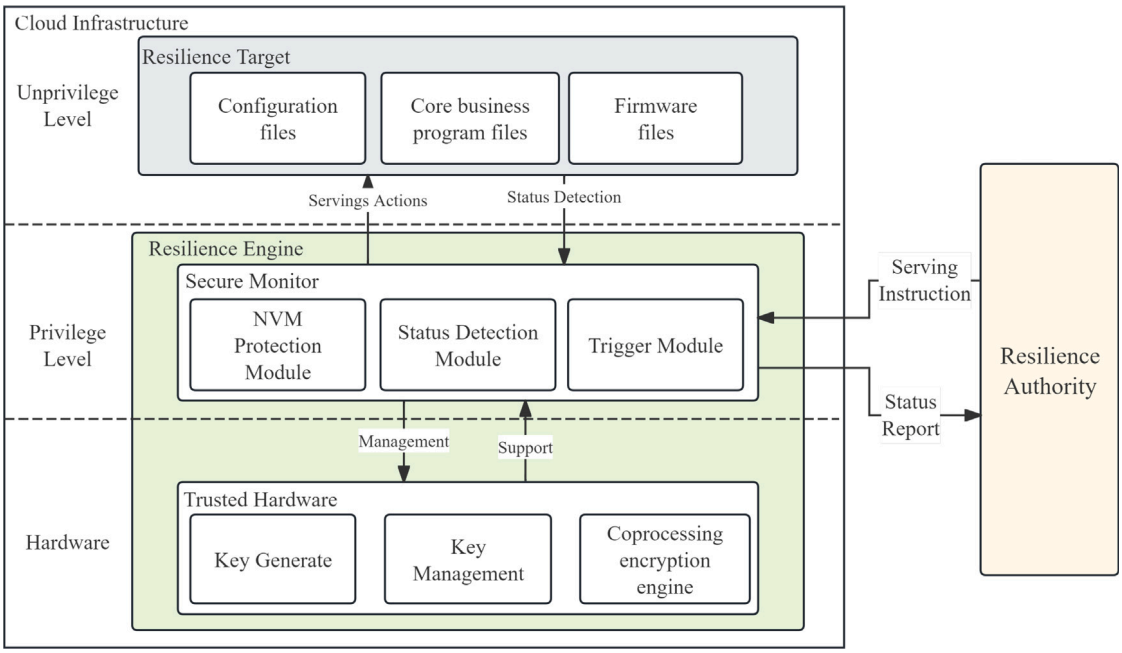


Fig. 1. TRCI framework.

Algorithm 1: Lock1:Security Baseline Protection Mechanism

Input: *ProtectedList* ; {A set of absolute paths for protected files}
Output: Action Status (**ALLOWED** or **DENIED**)
{Continuous monitoring loop }
while ***true*** **do**
  syscall ← WAIT_FOR_SYSCALL_HOOK(); NVPM_ENFORCE(*syscall*, *ProtectedList*);
Function NVPM_ENFORCE*(syscall, ProtectedList)*
  SensitiveOps ← {`openat`, `unlinkat`, ..., `truncate`};
  type ← GET_TYPE(*syscall*);
  if *type* ∈ *SensitiveOps* **then**
    path ← GET_PATH_FROM_PARAMETERS(*syscall*);
    if *path* ∈ *ProtectedList* **then**
      LOG_EVENT("Access Denied on " + *path*);
      BLOCK_AND_RETURN_ERROR(*syscall*);
      return DENIED; ; {Action taken: Block}
  ALLOW_SYSCALL_EXECUTION(*syscall*);
  return ALLOWED; ; {Action taken: Permit}

The ProtectedList is a manifest that enumerates the absolute paths of the Security-sensitive Baseline Files (SBF) designated for protection. By intercepting system calls, the Non-Volatile Memory Protection Module (NVPM) obtains the context of file operations and consults the ProtectedList to determine if the target of a modification attempt is a protected SBF. This list is itself a protected asset, subject to stringent access controls that prohibit any unauthorized modifications.

Modifications to the ProtectedList are permissible if and only if the Resilience Engine (RE) receives a legitimate, authenticated directive from the Resilience Authority (RA). The RA programmatically controls the protection status of an SBF by dynamically adding or removing its entry from the ProtectedList. When protection for a specific SBF is disabled—meaning its path is not present in the ProtectedList—the RE is authorized to perform legitimate lifecycle management operations, such as modifications or updates, as directed by the RA. Conversely, when protection is enabled, the SBF falls under the direct purview of the NVPM, and any modification attempts originating from malicious entities will be intercepted and denied.

4.3. Lock2: Secure Execution Environment (SEE) mechanism

As defined in our threat model, the RT may be under adversarial control, necessitating a isolated execution environment to conduct trusted recovery. The Secure Execution Environment (SEE) Mechanism is the process responsible for creating this temporary, high-assurance "fortress" for the RE. The structured process of instantiating this environment is the "locking" operation of **Lock 2** (SEE Protection Lock).

The SEE establishment process is triggered by the Secure Monitor immediately upon validating an external reset directive from the RA or detecting a critical internal integrity failure . The process, formalized in Algorithm 2, proceeds in three distinct stages:

Stage 1: Seize and Solidify Execution Control. The first priority is to atomically seize CPU control from the potentially compromised RT. Control is transferred to the Secure Monitor's entry code via a non-maskable interrupt (NMI) or a secure trap, immediately elevating the RE to the highest system privilege (e.g., ARM EL3 or Secure Monitor Mode) (Huang et al., 2024). To prevent interference from RT-controlled device drivers, all maskable interrupts are disabled (e.g., via the `cpsid instruction on ARM` instruction). Concurrently, the Secure Monitor dispatches Inter-Processor Interrupts (IPIs) to all other CPU cores, compelling them to halt execution of RT tasks. The contexts of these tasks are explicitly discarded, as they are presumed malicious.

Stage 2: Construct Memory and I/O Safety Fence. With RT execution frozen, the RE redefines the system's execution view by loading a pre-configured "secure page table" into the CPU's page table base register (e.g., ARM TTBR0). This new table exclusively maps the RE's code, the SBF (as read-only), and a dedicated working memory region. All physical pages previously owned by the RT are marked as non-present, rendering the RT's address space invisible and inaccessible. A Translation Lookaside Buffer (TLB) flush is immediately performed to purge any cached, invalid translations. Simultaneously, the IOMMU (e.g., ARM SMMU) is re-programmed to enforce a default-deny DMA policy, invalidating all RT-configured I/O mappings. A single, narrow exception is created to permit read-only DMA from the non-volatile

Algorithm 2: Lock 2: Secure Execution Environment (SEE) Mechanism

Input: None
Output: Initialization Status (**SUCCESS**)
Function ESTABLISH_SEE()
  {Stage 1: Seize and Solidify Execution Control }
  TRIGGER_SECURE_TRAP(); ; {Gain highest privilege}
  DISABLE_MASKABLE_INTERRUPTS(); BROADCAST_IPI_TO_HALT_CORES();
  {Stage 2: Construct Memory and I/O Safety Fence }
  LOAD_SECURE_PAGE_TABLE(*SEE_Page_Table_Addr*); FLUSH_TLB();
   PROGRAM_IOMMU(*Default_Policy*, *SBF_Read_Allow_Rule*);
  {Stage 3: Sanitize and Prepare Recovery Environment }
  ZERO_GENERAL_PURPOSE_REGISTERS(); INITIALIZE_SECURE_STACK();
  if ¬ VERIFY_INTEGRITY(*Recovery_Code_Addr*) **then**
    HALT_SYSTEM("Recovery code integrity failure");
  return SUCCESS; ; {Lock 2 is now engaged}

storage housing the SBF, strictly to the RE's private work buffer, thereby thwarting DMA-based attacks.

Stage 3: Sanitize and Prepare Recovery Environment. Finally, the Secure Monitor sanitizes the hardware state. All general-purpose registers are zeroed to purge residual data from the compromised RT, and a new, clean stack pointer is established. Before transferring control, the integrity of the recovery logic itself is verified against its baseline.

The SEE establish algorithm is detailed in Algorithm 2. Upon the completion of these sequential steps, Lock 2 is considered fully engaged. The system is in a state where the RT is inert, memory and I/O are fully isolated, and the RE possesses sole, verified control to proceed with the recovery.

4.4. Resilience trigger mechanism

The primary responsibility of the Status Detection Module is the continuous integrity measurement of the Resilience Target (RT), encompassing critical firmware, system configuration files, and business applications. This module employs cryptographic hash functions to validate data integrity, a method that directly corresponds to the "Tampering Attack" scenario defined in our threat model. The module leverages system call (syscall) hooks for its operation. When the RT attempts to launch or load a monitored component, such as executing an application, the syscall triggers a hook, enabling the module to preemptively perform an integrity measurement. The result is then compared against the security baseline, and reports detailing the target's state are generated for evaluation by the remote Resilience Authority (RA).

Following the RA's evaluation of the device state, the Resilience Trigger Module is responsible for accepting RA directives, activating the RE, and initiating the recovery process. In an ideal cooperative model, a simple software signal or a command from the RA would suffice to trigger a reset.

However, as defined by our threat model B2, a maliciously controlled RT possesses control over the system during normal operation. It therefore possesses the capability to intercept, delay, or spoof a simple reset signal, thereby nullifying the resilience mechanism. To counter this threat, the RE must possess the ability to forcibly seize device control even when the RT is non-cooperative or actively adversarial. To implement this forced-trigger capability, we employ a "watchdog" mechanism based on an Authenticated Timer (Algorithm 3). This timer is configured to force a system-wide reset after a fixed time period unless it receives an authenticated Deferral Ticket from the RA; the security of this mechanism is analyzed in Section 5. To ensure the integrity of this trigger mechanism itself, the executable files for the Authenticated Timer Signal Trigger are also stored in non-volatile memory and are similarly protected by the NVPM (Lock 1).

Algorithm 3: Resilience Trigger Mechanisms

Input: T_{reset} ; {The fixed time period for the hardware watchdog}
Output: Exit Status (**RECOVERY_INITIATED**)
Function AUTHENTICATED_TIMER_TRIGGER()
  while ***true*** **do**
    {Block until ticket arrives or timer expires }
    $ticket \leftarrow$ WAIT_FOR_TICKET_OR_TIMEOUT(T_{reset});
    if $ticket == null$ **then**
      {Timer expired, no valid ticket received }
      INITIATE_RECOVERY(); ; {Forcibly transfer control to RE}
      return RECOVERY_INITIATED; ; {Exit loop to handle reset}
    else
      {A ticket was received, must verify }
      if VERIFY_RA_SIGNATURE(*ticket*) == ***true*** **then**
        KICK_WATCHDOG(); ; {Defer reset for another cycle}
      else
        LOG_EVENT("Invalid/Forged ticket ignored");
        {Do not kick, let timer expire and reset }

4.5. Resilience recovery methods

As defined in the threat model (**Security Boundary B1**), a Secure Execution Environment (SEE) is required to ensure that the trusted recovery process executes as intended, protected from potential interference by a Runtime Environment (RT) which may be under adversarial control. To formally delineate this resilient recovery mechanism, we classify the resources within the edge infrastructure into two distinct categories:

SEE Resources: These resources, which encompass the previously described Secure Boot File (SBF) as well as the code segments, memory regions, registers, and computational cores dedicated to recovery operations, are accessible only within the SEE. During critical recovery phases, these resources are exclusively allocated to the Recovery Engine (RE) and must be protected by stringent access isolation measures to prevent any interference from the RT.

Ordinary Resources: This category includes all other resources that support general-purpose workload operations.

Notably, certain SEE resources, such as a subset of CPU cores and registers, may also be accessible to the RT under normal operating conditions. However, once the SEE is instantiated and prior to the commencement of recovery, any such access by the RT is systematically revoked.

To ensure the recovery process proceeds in a trustworthy manner, our scheme introduces a two-lock mechanism governing the SEE construction:

Lock 1: SBF Protection Lock. This lock is enabled by default upon system boot and is enforced via the kernel protection mechanism detailed in Section 4.2. During normal system operation, the non-volatile storage containing the golden image and measurement baseline is configured as read-only by the Secure Monitor. Any write attempt, whether originating from the RT's kernel or user space, is denied and raises an exception, which is subsequently handled by the Secure Monitor as a security event. Only the Secure Monitor itself is permitted to modify this configuration, and only after it has entered the SEE.

Lock 2: SEE Protection Lock. This lock is initially disengaged following the system boot. Its "locking" operation corresponds to the structured process of establishing the SEE, whereas "unlocking" refers to the controlled teardown of the SEE and the subsequent return of control to the RT.

Table 1
Holistic security mechanism phases and actions.

Phase	Status	Implementer	Action	Lock
Phase 0: Normal operation	Continuous monitoring	RE	Periodically measures RT integrity, compares against TPM baseline, and sends signed heartbeats to RA.	Lock1: ON Lock2: OFF
Phase 1: Anomaly detection	Anomaly detected	RE & RA	Internal (RE) or external (RA) anomaly detected (e.g., hash mismatch or lost heartbeat).	Lock1: ON Lock2: OFF
Phase 2: Instruction issuance and verification	RA sends reset instruction	RA → RE	RA sends signed "Reset" instruction via encrypted tunnel; RE verifies the signature.	Lock1: ON Lock2: OFF
Phase 3: Establish SEE	Activating environment lock (Lock 2)	RE	RE freezes RT, isolates its memory (MMU), and blocks peripheral DMA (IOMMU).	Lock1: ON Lock2: TURNING ON
Phase 4: Execute recovery	Unlock Lock1 → Recover → Relock Lock1	RE	Inside SEE: Temporarily unlock Lock 1, overwrite RT with golden image, then immediately re-lock Lock 1.	Lock1: OFF Lock2: ON
Phase 5: Return control	Disengaging environment lock (Lock 2)	RE	RE tears down SEE (restores MMU/IOMMU), reboots system into recovered RT, and reports success.	Lock1: ON Lock2: TURNING OFF

Table 1 details the lifecycle of our holistic security mechanism, deconstructing it into six distinct phases (Phase 0 - Phase 5) from normal operation to full recovery. This process illustrates the interactions among the Resilience Engine (RE), the Resilience Authority (RA), and the Runtime Target (RT), as well as the state transitions of the two core locks (Lock 1 and Lock 2).

Phase 0 (Normal Operation) is the system's steady state. During this phase, the RE (Secure Monitor) continuously performs background integrity monitoring. This includes: 1. Periodically computing the hash values of critical RT components (e.g., the kernel image); 2. Comparing these hashes against the secure "measurement baseline" stored within the TPM; and 3. Transmitting signed heartbeat packets and platform attestation reports to the RA to certify its health and integrity.

Phase 1 (Anomaly Detection) serves as the trigger point for the recovery process. This trigger can originate from two scenarios: (A) Internal Discovery, where the RE's Secure Monitor detects a hash mismatch and immediately reports the alert; or (B) External Discovery, where the RA identifies anomalous device behavior via encrypted tunnel and determines to initiate a reset.

Phase 2 (Instruction Issuance and Verification) is the RA's responsive action. The RA generates an instruction packet containing the "Reset" command, which is digitally signed using its private key. This instruction is transmitted via the encrypted tunnel to the target device's RE. Upon receipt, the RE validates the signature's authenticity and integrity using the pre-stored RA public key to prevent forged reset attacks.

Phase 3 (Establish SEE) is the first step of the recovery action, corresponding to the activation of Lock 2. Following instruction validation, the RE immediately executes a sequence of isolation procedures: 1. It freezes the RT by halting the OS scheduler; 2. It isolates RT memory by rewriting the MMU page tables; and 3. It blocks all unauthorized DMA by configuring the IOMMU, thereby establishing a robust "safe fortress" for the recovery process.

Phase 4 (Execute Recovery) is conducted entirely within the established SEE. Here, the temporal logic of the dual-lock mechanism is demonstrated: Lock 1 is temporarily disengaged, permitting the RE write-access to the non-volatile storage containing the "golden image". The RE then executes the overwrite operation, performing a bit-for-bit copy of the golden image onto the compromised RT memory partitions. Upon completion and verification, Lock 1 is immediately re-engaged, returning the security baseline to its read-only state.

Phase 5 (Return Control) involves the safe teardown of the SEE, or the disengagement of Lock 2. The RE restores the normal IOMMU and MMU configurations and then triggers a system-level restart. The Bootloader is directed to load the freshly recovered, clean RT operating system. After startup, the RE resumes its background monitoring role and reports the successful recovery to the RA, completing the entire closed-loop process. A Finite-State Machine Diagram (Fig. 2) illustrates the control states of recovery.

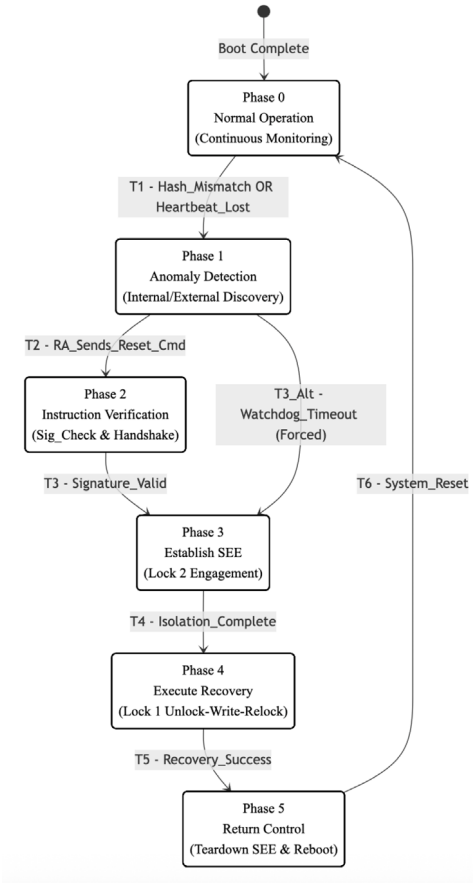


Fig. 2. The control states of recovery.

4.6. Secure authorized update workflow

While the recovery mechanism detailed in Section 4.5 restores the system from a local golden image, edge devices also require dynamic updates from the Resilience Authority (RA) to patch vulnerabilities or upgrade firmware. Temporarily lifting file protections to apply these

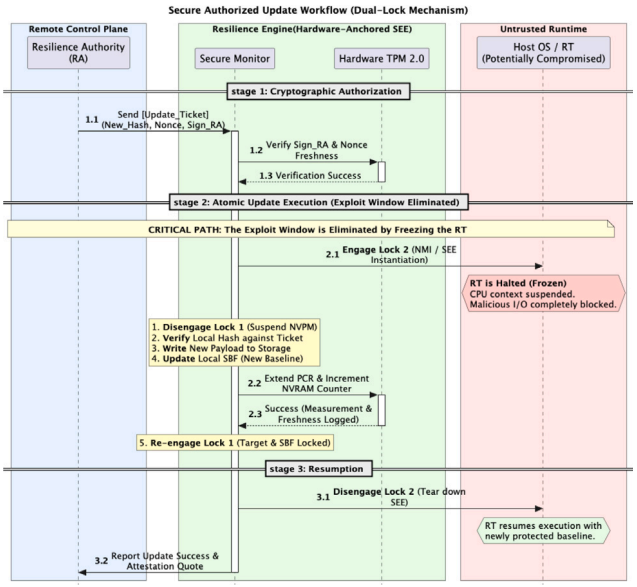


Fig. 3. Update Workflow.

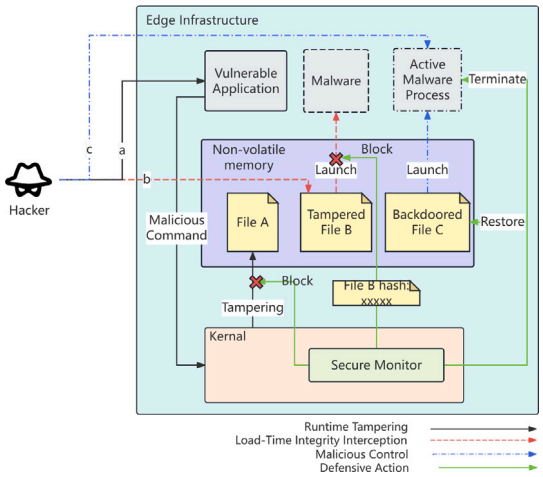


Fig. 4. Defense against the RT attack surface.

updates, however, creates a "Time-of-Check to Time-of-Use" (TOCTOU) transitionary window. An attacker with sufficient privileges could exploit this brief state to interfere with the update process.

To manage this vulnerability, TRCI structures a legitimate update as a cryptographically bound, atomic operation that reuses the Dual-Lock architecture. The secure update workflow, illustrated in Fig. 3, proceeds as follows:

Stage 1: The RA issues an Update_Ticket containing the new payload's hash, a monotonic nonce, and the RA's digital signature. The Resilience Engine (RE) verifies this signature and nonce freshness via the hardware TPM to confirm authenticity and prevent replay attacks.

Stage 2: Rather than immediately disengaging the file lock (Lock 1) upon ticket verification, the RE mirrors the recovery procedure by first engaging Lock 2 (SEE Instantiation). The resulting NMI suspends the execution context of the Runtime Environment (RT). While the RT is paused, the RE temporarily disengages Lock 1 to perform the internal update. First, the RE locally computes the hash of the downloaded payload and strictly verifies this digest against the RA-signed hash contained within the Update_Ticket to ensure absolute transit integrity. Only after this verification succeeds does the RE write the new payload to storage and update the local Secure Boot File (SBF) to establish the new baseline. Following these writes, the RE extends the new hash into the designated TPM PCR, logging the update event for future remote attestation. To guarantee unambiguous replay and rollback protection, the framework's freshness mechanism relies on a hardware monotonic counter located in the TPM's Non-Volatile RAM (NVRAM). By storing only this minimal state anchor in the hardware, the design avoids TPM storage constraints while protecting the dynamic payload. The counter is atomically incremented only after the new baseline is successfully committed, thereby explicitly invalidating prior tickets. Finally, Lock 1 is re-engaged to secure both the target and the SBF.

Stage 3: Once the files are secured against runtime modifications, Lock 2 is disengaged, tearing down the SEE. The RT resumes execution under the updated security policies, and the RE reports the successful update along with an attestation quote to the RA.

5. Security analysis

This section comprehensively analyzes the system design proposed in Section 4.1, evaluating its security capabilities under the threat model defined in Section 3. We systematically assess how the architecture defends against these threats by addressing the two primary attack surfaces and the three core attack methods: Tampering Attack, Backdoor Implantation, and Refusal of Execution.

5.1. Defense against the RT attack surface (Boundary B1)

Tampering Attack and **Backdoor Implantation** are two threats defined in our threat model, wherein an adversary, after gaining system access, attempts to modify the RT (e.g., critical firmware or business application files).

This architecture provides a two-layered, defense-in-depth approach to counter such attacks:

1. Runtime Tampering Defense: First, an adversary might attempt to directly tamper with the RT files in non-volatile storage. As shown in Fig. 4 attack path a, our design provides active defense via the **Lock 1** (NVPM) mechanism (Section 4.2, Algorithm 1). The Secure Monitor, operating at a privileged level, intercepts all write operations (e.g., `openat`, `unlinkat`) from the RT via system call (syscall) hooks. Any malicious tampering attempt targeting a protected file in the `ProtectedList` (i.e., an RT component) is identified by the NVPM and immediately denied (`Block_and_Return_Error`). Therefore, during normal system operation, the RT cannot tamper with its own disk image through standard I/O paths.

2. Load-Time Integrity Interception: Assuming an adversary uses advanced techniques (e.g., a kernel-level rootkit exploiting a 0-day vulnerability, or an offline DMA attack on the edge device) to successfully bypass Lock 1's real-time interception and successfully implants a backdoor (as shown in Fig. 4 path b) into the RT file on non-volatile storage. In this scenario, the Status Detection Module (Section 4.3) comes into play. When the RT attempts to launch or load this tampered component, the syscall triggers a hook. This module preemptively performs an integrity measurement (computing a hash) on the file and compares it against the "golden measurement" stored in the SBF and protected by Lock 1. As the file has been tampered with, the computed hash will not match the baseline. The Status Detection Module immediately identifies this integrity violation, prevents the loading and execution of the component, and reports this "Internal Discovery" anomaly (Phase 1) to the RA. Therefore, even if the tampering is successful, the backdoor cannot be activated, and the attack is intercepted before execution.

3. Authorized Update Window Defense: While Lock 1 prevents unauthorized tampering during normal operations, dynamically updating the RT (Section 4.6) introduces a potential "Time-of-Check to Time-of-Use" (TOCTOU) vulnerability window if protections are temporarily paused. An attacker with Ring-0 privileges might attempt

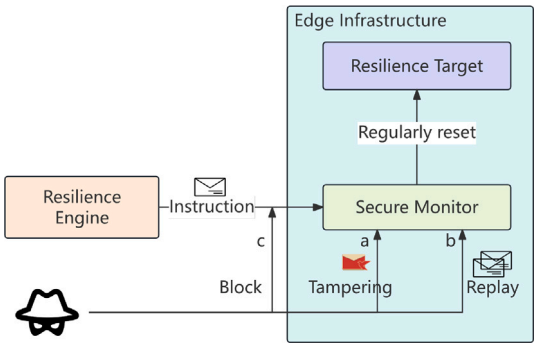


Fig. 5. Defense against communication channel attacks.

to leverage this transitionary state to execute a tampering attack (A1) or implant a backdoor (A2).

TRCI neutralizes this specific edge-case vector by enforcing atomic execution through the Dual-Lock architecture. Remote impersonation of an update is inherently prevented by the cryptographic ticketing mechanism (analyzed in Section 5.2). More critically, local preemption is precluded by engaging Lock 2 (SEE) prior to disengaging Lock 1. Because the compromised OS is strictly halted via an NMI, it is starved of CPU cycles and cannot schedule malicious I/O operations to exploit the brief unlocked window. The entire update and measurement sequence is completed atomically before the RT is allowed to resume, effectively sealing the TOCTOU gap within the established Boundary B1 threat model.

In addition, a more advanced adversary might attempt to evade all static and load-time detection. For example, by using 0-day vulnerabilities, "fileless" attack techniques, or exploiting memory vulnerabilities to implant malicious code directly into the memory space of a running RT process, without modifying any files on disk (as shown in Fig. 4 path c). In this situation, although load-time detection may be bypassed, the system's continuous monitoring and forced-trigger mechanisms remain effective. For instance, a malicious RT running in memory may cause observable anomalies; it might interfere with the Status Detection Module to hide itself, causing the periodic health reports (Phase 0) sent to the RA to be interrupted or contain anomalous data. Once the RA detects such an anomaly (i.e., Phase 1B), it will cease sending "Deferral Tickets".

Regardless of how the infected RT attempts to maintain its operation, the hardware watchdog of the Authenticated Timer (Algorithm 3) will time out after the T_{reset} period due to the absence of a valid ticket. This triggers an un-blockable, hardware-level reset, forcibly transferring device control from the RT to the RE, which then initiates the SEE establishment and full recovery process described in Section 4.5. Thus, even if an adversary bypasses load-time detection, their malicious activity is discovered by the RA due to the compromised trusted state, and is ultimately remediated by the forced recovery mechanism.

5.2. Defense against communication channel attacks (Boundary B2)

The B2 attack surface concerns the communication channel between the RE and the RA. We analyze three primary channel attacks:

1. Command Tampering: An adversary (e.g., Man-in-the-Middle, MitM) intercepts and modifies a legitimate "Deferral Ticket" sent by the RA, or forges a "Reset" command (as shown in Fig. 5 path a). Our design (Section 4.4) dictates that all RA-RE communications (including tickets and directives) must be digitally signed using the RA's private key. The RE (Resilience Trigger Module) verifies the signature of every packet using the pre-stored RA public key. Any tampering invalidates the signature, causing the packet to be discarded as invalid by the RE.

2. Command Replay: An adversary (as depicted in Fig. 5 path b) records a "Deferral Ticket" captured during a past legitimate session (e.g., when the network was healthy) and "replays" this old ticket when the RA has decided to stop sending tickets to trigger a reset (e.g., during an anomaly), attempting to deceive the RE into deferring the reset. Our secure channel establishment (Section 4.4) employs a Challenge-Response handshake protocol based on public-key signatures. This protocol ensures "freshness" by exchanging signed nonces. The Resilience Trigger Module will reject any packet with an expired or previously used nonce/seq. As a result, replayed old tickets are identified and discarded, and cannot prevent the legitimate reset process.

3. Command Interception/Refusal of Execution: This is the most severe attack scenario on the B2 plane defined in our threat model: a fully compromised RT is in a non-cooperative state and actively resists recovery(as depicted in Fig. 5 path c). For instance, an adversary successfully intercepts and discards "Deferral Tickets" sent by the RA, or attempts to prevent the RE from deferring the reset, thereby (erroneously) triggering a DoS attack.

This architecture ensures forced recovery through two key designs: First, the **Un-blockable Trigger**: The Authenticated Timer mechanism (Section 4.4, Algorithm 3) provides a hardware trigger independent of the RT's software state. The RT can neither forge a signed Deferral Ticket nor prevent recovery by intercepting tickets. When the RA ceases authorization (stops sending tickets), a reset is inevitable.

Second, the **Un-interruptible Execution**: Merely triggering a reset is insufficient; a malicious RT might attempt to interfere with the recovery process after the reset signal is issued but before the RE has taken full control. Our **Lock 2 (SEE Mechanism)** (Section 4.3, Algorithm 2) solves this problem. The establishment of the SEE is not a software request but a control-seizure action (Stage 1) implemented via NMI or a secure trap, forcibly freezing all RT tasks on other CPU cores and discarding their (untrusted) contexts. It completely severs all RT access to memory and I/O via the MMU and IOMMU (Stage 2).

In summary, this design ensures that the entire chain, from "trigger" to "execution", does not depend on any cooperation from the RT, thereby achieving forced recovery even in the worst-case scenario (a fully malicious RT).

5.3. Formal analysis of recovery liveness

To rigorously demonstrate the deterministic nature of our recovery mechanism, we abstract the operational lifecycle detailed in Table 1 into a Finite State Machine (FSM) model. This abstraction allows us to mathematically prove the **Liveness Property**, demonstrating that the transition to the recovery state is inevitable even under a *Refusal of Execution* attack (Threat A4).

Definition 1 (*System Model*)**.** Let the system be denoted by the tuple $\mathcal{M} = \langle S, E, \delta, \mathcal{T} \rangle$. The state space S maps directly to the operational phases: S_{safe} corresponds to the trusted normal operation (Phase 0); S_{vuln} represents the vulnerable states where an anomaly is flagged but remediation is pending (covering Phase 1 and Phase 2); and S_{rec} abstracts the atomic recovery sequence (Phases 3–5) where **Lock 2** is engaged. Most importantly, we define $S_{mal} \subset S_{vuln}$ as the non-cooperative state where a compromised RT actively blocks interrupts or drops RA commands.

The system dynamics are driven by a set of events E. Normal operation relies on e_{tick} (a valid "Deferral Ticket" verified by hardware) to reset the watchdog timer $\mathcal{T}$. The critical transition relies on e_{tout} (timer expiry), which triggers the transition rule $\delta(S_{any}, e_{tout}) \rightarrow S_{rec}$. This hardware-enforced rule overrides all software states.

Theorem 1 (*Inevitability of Recovery*)**.** *For any initial state $s_0 = S_{mal}$, there exists a finite time t such that the system state $s_t = S_{rec}$.*

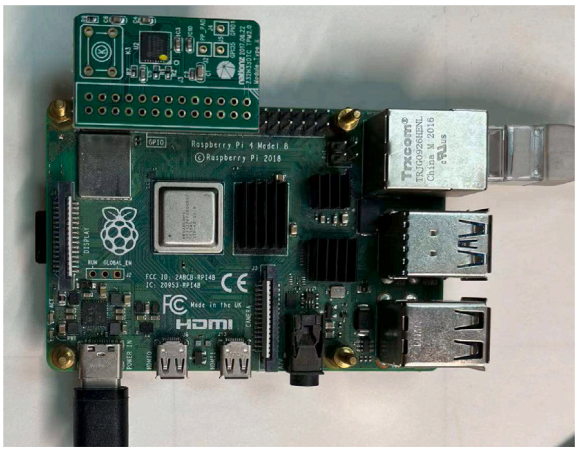


Fig. 6. Deployed device.

Proof. We proceed by contradiction. Assume the system can remain in the non-cooperative state S_{mal} indefinitely ($t \to \infty$). For this condition to hold, the hardware watchdog $\mathcal{T}$ must be periodically reset, which requires the continuous generation of e_{tick}. However, according to the protocol, e_{tick} is issued by the RA only upon receipt of a valid device attestation (e_{valid}).

In the state S_{mal}, the adversary controls the RT but faces a dilemma. To generate e_{valid}, the adversary must produce a valid signature. Due to the cryptographic binding enforced by Lock 1 (NVPM), forging this signature without the hardware-protected private key is computationally infeasible (Case A). Alternatively, if the adversary attempts a Denial-of-Service by blocking communications (Threat B3), the RA receives no attestation (Case B).

In both cases, the condition for generating e_{tick} is violated. Thus, the hardware timer $\mathcal{T}$ decreases monotonically. Upon expiry ($t = T_{reset}$), the event e_{tout} triggers the forced transition $\delta(S_{mal}, e_{tout}) = S_{rec}$. This transition is implemented via a Non-Maskable Interrupt (NMI) or hardware reset line, which cannot be intercepted by the compromised software in S_{mal}. Thus, the assumption of indefinite persistence in S_{mal} is false, and recovery is proven to be deterministic. □

6. Implementation and evaluation

The evaluation is structured around two distinct hardware environments to separately assess comparative performance and architectural scalability.

The primary evaluation (Sections 6.1 and 6.2) is conducted on a resource-constrained ARM edge node. This environment is intentionally selected to amplify the overhead sensitivities, providing a clear horizontal comparison among different defense mechanisms under restricted computational and I/O capacities.

Subsequently, to isolate and analyze the specific hardware bus bottleneck (I2C) observed in the primary testbed, Section 6.3 presents a supplementary hardware ablation study. This secondary evaluation utilizes an x86 industrial gateway profile to evaluate the vertical scalability of the TRCI architecture when supported by high-speed hardware components, focusing on the latency of the cryptographic anchoring mechanism.

To execute the primary evaluation, we established a representative, general-purpose edge testbed. As illustrated in Fig. 6, this host device utilizes a Raspberry Pi 4B interconnected with a NationZ TPM 2.0 chip (z32h330tc) via the I2C bus. The system operates on CentOS 8.4, configured with tpm2-tools (v4.1.1–5.el8) and tpm2-abrmd (v2.3.3–2.el8) to ensure effective TPM utilization in kernel mode.

Cryptographic functionalities are provided through APIs exposed by the hardware TPM. It is noted that the implementation of system call hooking may vary across kernel versions; the CentOS 8.x distribution used in this study is based on kernel version 4.18.0 or later.

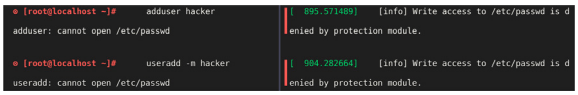


Fig. 7. Tampering block.

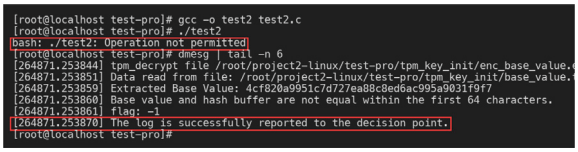


Fig. 8. Booting block.

The selection of this testbed is predicated on its role as a representative, general-purpose model, which allows for a robust evaluation of our method's broad applicability. This platform effectively integrates three critical, standardized components prevalent in diverse, modern edge infrastructures: (1) the ARMv8 architecture, which is the dominant architecture for high-performance edge gateways and IoT devices; (2) a standard, server-grade Linux Kernel (4.18+), ensuring that our kernel-mode mechanisms (e.g., system call hooking, module loading via insmod) are directly portable across the vast ecosystem of Linux-based platforms; and (3) a TCG-compliant TPM 2.0, which demonstrates that our solution interfaces with the standardized hardware root of trust (via tpm2-tools) rather than a proprietary, vendor-specific implementation. Success on this testbed thus provides high confidence in the solution's portability and general applicability to various edge computing platforms.

6.1. Functional verification

To validate the efficacy of TRCI, we conducted a systematic verification against the attack vectors defined in our Threat Model (Section 3).

6.1.1. Defense against data plane attacks (Boundary B1)

The first set of experiments addresses attacks on the B1 boundary.

An adversary may leverage a vulnerable application or introduce a malicious one to modify the golden measurement baseline itself, attempting to bypass the load-time integrity check. Alternatively, they may modify critical configuration files for privilege escalation. In our experiment, we protect /etc/passwd to block this key step. As shown in Fig. 7, the NVPM (Lock 1) successfully intercepts and denies write access, causing both adduser and useradd commands to fail. The kernel log confirms, "Write access to /etc/passwd is denied by protection module". confirming that **A1** is effectively blocked at runtime.

We replaced a legitimate binary with a tampered version (./test2) to simulate a backdoor implantation. When an adversary tampers with business systems or firmware and attempts to (re)start them, the RE intercepts the action. As shown in Fig. 8, the Status Detection Module calculates the file's hash upon the execution attempt. It finds that the "Base value and hash buffer are not equal", and denies the execution request (bash: Operation not permitted). The integrity failure log is then "successfully reported to the decision point" (the RA). This confirms the defense against **A2**.

To simulate an advanced in-memory attack (**A3**) that leads to a non-cooperative state (**A4**), we injected a process that hangs the OS and blocks all software-level signals. When the RA detects an anomaly and issues a RESET command to recover a file (e.g., test.txt), the malicious RT could attempt to obstruct the RE's service. As demonstrated in Fig. 9, our mechanism counters this: the RE successfully establishes the SEE (engaging Lock 2), seizes control, and performs

```
Secure Monitor: Message signature verification successful
Secure Monitor: RESET command
Secure Monitor: Resetting file test.txt
Secure Monitor: get lock
Secure Monitor: invoke protection module to disable file protection...
Secure Monitor: Sending message to kernel: del /home/using/test.txt
Secure Monitor: Received message payload: Command processed
Secure Monitor: Sending message to kernel: add /home/using/test.txt
Secure Monitor: Received message payload: Command processed
Secure Monitor: Resetting file some.c
Secure Monitor: get lock
Secure Monitor: invoke protection module to disable file protection...
Secure Monitor: Sending message to kernel: del /home/using/some.c
Secure Monitor: Received message payload: Command processed
Secure Monitor: Sending message to kernel: add /home/using/some.c
Secure Monitor: Received message payload: Command processed
Secure Monitor: Waiting for trigger module command...
```

Fig. 9. Anti-interference execution.

```
FB 9B 2D 15 95 63 4F 72 4E A3 BC C6 1A AA DC 01
35 97 89 3F 69 1A FC 39 FE A2 3E 70 BA 2F D0 27
FF DE DF 00 B9 D6 42 8B 29 DE D0 A8 06 33 E7 57
6F D3 09 C2 C3 C5 0E 21 C9 D3 67 D0 FD E6 5A 95
FA 8F 33 23 E4 5B 86 0E 07 35 8F 2A 54 FB 05 94
6A 30 5D FF BF 41 7E 1F 34 91 F6 0D 8A F2 D8 61
FF 6D 20 EC 68 1F 94 05 2C BD 00 86 93 51 9A E3
7F 00 00 00
Signature verification failed, message has been modified
Sending delay
```

Fig. 10. Forge key attack.

```
00 53 A9 AF 94 C1 AC 60 FF 8C 86 27 64 38 13 50
E1 28 7A E0 68 14 BD 28 A7 BF 13 4D 9A D6 C3 FC
33 42 62 CF C1 33 76 31 C2 7E 56 2A 77 CE 1C A3
75 01 9C 59 5F 7E E5 37 A8 BA 55 47 66 98 CF 68
FE 88 57 4E 11 57 A7 D6 78 1C 56 51 2B 64 7B BB
27 93 03 2C 96 3B FE 20 13 A9 36 0F 20 7D FF CE
2F 9F EB F0 34 F0 62 32 57 BF 8D 1D 8A 40 84 D9
1D AB ED B5 7D 8A C2 70 97 79 C7 F7 0C DA E7 8E
2C 00 00 00 Signature verification successful
Same Sequence;Replay attack detected
Sending delay
```

Fig. 11. Replay attack.

```
07 FD 40 EE 39 8E C6 E7 C0 E1 F9 DE 03 A2 B2 CC
FA 00 00 00
Start timing, stop sending delay signal if no reply is received within 3 seco
!
Sending delay
Sending delay
Stop sending delay signals to the Watch dog
Reply : ack=0, seq=0
Received message:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Fig. 12. Deferral tickets block.

the trusted recovery. This involves invoking the protection module to "get lock" (disabling Lock 1), deleting the compromised file (del /home/using/test.txt), and restoring the golden version (add /home/using/test.txt).

This multi-layered defense is effective against real-world attack vectors. While incidents like the MikroTik (Muin et al., 2022) and Krebs on Security breaches (Gyányi, 2024) rely on persistent configuration file tampering (A1), commercial edge gateways remain equally susceptible to memory corruption exploits. For instance, a recently disclosed stack buffer overflow vulnerability in enterprise 5G PoE routers (CNVD-2026-10807 China National Vulnerability Database (CNVD), 2026) allows attackers to trigger a local denial-of-service and fatal system crashes. Such exploits directly induce the non-cooperative state (A4) defined in our threat model, where pure software-based remediation fails. This underscores the operational necessity of TRCI's hardware-triggered SEE (Lock 2) for autonomous recovery. As summarized in Table 2, when compared to inflexible, ROM-based solutions, TRCI provides better flexibility while guaranteeing deterministic restoration.

6.1.2. Defense against control plane attacks (Boundary B2)

The second set of experiments addresses attacks on the B2 (RE vs. RA/Network) boundary.

First, we test against a malicious RT command forgery attack (**N1**). As the adversary does not possess the RA's private key, the forged directive fails the cryptographic signature verification. As shown in Fig. 10, the RE receives the tampered packet, and the log clearly indicates "Signature verification failed, message has been modified", causing the packet to be discarded.

Second, we test against a malicious RT command replay attack. Here, the adversary captures and replays a legitimate, signed command to bypass verification (**N2**). As shown in Fig. 11, this attack is defeated at the protocol level. The RE's secure channel relies on a challenge-response handshake with signed nonces (or sequence numbers, seq). The replayed packet's seq number is invalid or out-of-order (e.g., "Connection Request: SYN = 0,seq = 0"), causing the RE to reject it as a replay attack even if the signature itself is valid.

Finally, we test against a malicious RT intercepting commands (**N3**). The adversary blocks all incoming "Deferral Tickets", attempting to prevent RA commands from reaching the RE. As shown in Fig. 12, this attack is rendered ineffective by the Authenticated Timer (watchdog) design. The RE, having not received the delay signal within the specified 3-second window, "Stop sending delay signals to the power manager". The hardware timer expires, triggering the forced recovery process as designed.

By securing the communication channel, TRCI effectively neutralizes command injection and replay vectors, addressing the remote control vulnerabilities often exploited in large-scale botnet incidents.

The analysis in Table 2 highlights the significant gaps in existing approaches and the comprehensive nature of our design:

Static Hardware PFR (NIST SP 800–193, Intel PFR): This category represents the industry standard for firmware resilience. While it provides excellent, non-bypassable hardware protection against direct tampering and boot-time backdoors (A1, A2), its fundamental limitation is its inflexibility (F1). It is designed to protect static firmware, not the dynamic configuration files or business applications (like /etc/passwd or test.txt) that are the subject of our work. It offers no defense against in-memory/fileless attacks (A3) or non-cooperative refusal-of-execution (A4).

Software FIM (Tripwire, AIDE): These solutions are purely detective, not preventative. They can report on tampering (A1, A2) after it has already occurred, which is often too late. More critically, as they operate entirely within the RT (user or kernel space), they are rendered completely ineffective once the OS is compromised (A3, A4). An attacker with root privileges can simply circumvent or manipulate the FIM tool itself.

Measurement-Based Integrity (Linux IMA, Windows Measured Boot): This approach, built on Secure Boot and TPM, is a significant improvement. It provides load-time detection (A2) by measuring files before they are executed. However, it still has two major weaknesses addressed by our design: (1) It offers no runtime protection (A1) against attacks like the one on /etc/passwd (which is modified while the system is running). (2) It has no native mechanism to force recovery in a non-cooperative state (A4). Its focus is detection and attestation, not guaranteed remediation.

Hardware-Assisted Cross-Layer Defenses (Category 4): This category encompasses emerging paradigms that embed security logic closer to the hardware. Approaches utilizing HPCs and machine learning excel at detecting advanced in-memory anomalies (A3) by monitoring micro-architectural deviations natively. However, they function primarily as passive diagnostics; they cannot proactively block unauthorized file I/O operations (A1). Furthermore, while they provide robust detection, they generally lack an independent hardware trigger to force a system restoration if the compromised OS refuses to cooperate (A4). Conversely, cybersecure-by-design solutions that embed rollback mechanisms directly into custom processors (e.g., FPGA soft-cores) provide

Table 2
Comparative analysis of resilience methodologies against key attack vectors.

Solution/Category	A1: Runtime file tampering (e.g., Modify /etc/passwd)	A2: Load-time backdoor (e.g., Execute tampered binary)	A3: In-memory attack (e.g., Fileless malware)	A4: Refusal of execution (e.g., Block recovery signal)	F1: Flexible & secure update (e.g., Patch protected file)
TRCI (Our Method)	✓ (Blocked by **Lock 1**)	✓ (Blocked by **Status Detection**)	✓ (Remediated by **Auth. Timer**)	✓ (Forced by **Auth. Timer & Lock 2**)	✓ (RA-authorized)
Category 1: Static Hardware PFR					
NIST SP 800-193 (Bartock et al., 2018)	✓	✓	✗	✗	✗
Intel PFR (Regenscheid, 2017)	✓	✓	✗	✗	✗
Category 2: Software FIM (Detect-Only)					
Tripwire (Liu and Jiang, 2023)	~ (Detects, cannot block)	~ (Detects, cannot block)	✗ (Runs on RT)	✗ (Runs on RT)	✓ (N/A)
AIDE (Kwon et al., 2022)	~ (Detects, cannot block)	~ (Detects, cannot block)	✗ (Runs on RT)	✗ (Runs on RT)	✓ (N/A)
Category 3: Measurement-Based (Detect-at-Load)					
Linux IMA (Litchfield and Du, 2023)	✗ (No runtime prevention)	✓ (Detects at load-time)	✗ (No runtime detection)	✗ (No trigger)	~ (Breaks attestation)
Windows Measured Boot (Surve et al., 2023)	✗ (No runtime prevention)	✓ (Detects at boot-time)	✗ (No runtime detection)	✗ (No trigger)	~ (Breaks attestation)
Category 4: Hardware-Assisted cross-layer defenses					
HPC & ML Detection (Chenet et al., 2024; Foreman, 2018)	✗ (Diagnostic only, no I/O block)	~ (Probabilistic detection)	✓ (Dynamic tracing)	✗ (Lacks HW recovery trigger)	~ (Requires model updates)
Custom HW CFI & Recovery (DaSilva and Fortier, 2019)	✗ (Focuses on CPU flow, not files)	~ (Detects hijacked flow)	✓ (HW enforced isolation)	✓ (HW triggers rollback)	✗ (Rigid custom pipeline)

robust, non-cooperative recovery (A4). Nevertheless, their implementation fundamentally focuses on instruction-level execution rather than file system semantics (A1), and relying on customized silicon restricts their capability to securely manage dynamic artifacts and frequent system updates (F1) across standardized COTS hardware.

TRCI (Our Method): Our framework synthesizes the functional strengths of these domains to holistically implement the NIST cyber resilience triad (Protect, Detect, Recover) across clearly delineated system entities. By structurally decoupling the vulnerable execution environment (Resilience Target) from the isolated enforcement modules (Resilience Engine), TRCI clarifies operational security boundaries. Lock 1 (NVPM) establishes the proactive **protection** (A1) absent in measurement and execution-focused models. The Status Detection Module provides reactive **detection** at the load-time boundary (A2). Crucially, while our software-level monitoring shares similar kernel dependencies with other diagnostic tools, TRCI completes the resilience lifecycle by orchestrating standard COTS hardware (the Authenticated Timer and Lock 2) to serve as an out-of-band fail-safe. This guarantees a deterministic **recovery** trigger (A4) against advanced in-memory subversions (A3) even if the OS becomes non-cooperative, while preserving the flexibility required for authorized updates (F1).

6.2. Efficiency

To rigorously evaluate the performance overhead and scalability of TRCI, we conducted a comprehensive series of experiments on the testbed described in Section 6.1. Our evaluation compares TRCI against a standard Native Linux baseline and two state-of-the-art solutions: **MS-IDS**(van Sloun et al., 2025) representing a high-performance kernel-level monitoring solution, and **LAKEP**(He et al., 2023), representing a lightweight IoT communication protocol.

6.2.1. Steady-state system overhead

We first evaluated the "silent cost" of the system under Phase 0 (Normal Operation), where the Resilience Engine performs passive monitoring. As illustrated in Fig. 13, we measured the CPU and Memory usage on a Raspberry Pi 4B platform under idle conditions.

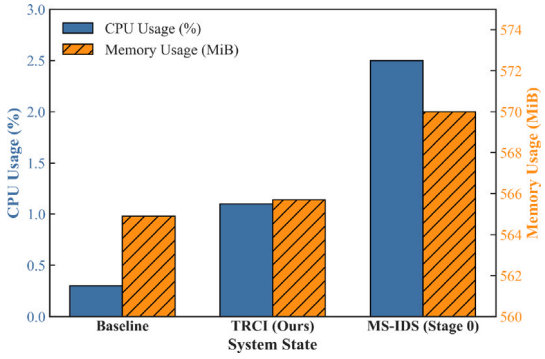


Fig. 13. Resource overhead.

The experimental results indicate that TRCI introduces a marginal overhead compared to the Native Linux baseline. Specifically, CPU usage increased from 0.30% (Baseline) to 1.10% (TRCI), resulting in a Δ of +0.80%. Similarly, memory consumption saw a negligible increase of approximately 0.8 MiB (from 564.9 MiB to 565.7 MiB).

In comparison, the MS-IDS solution (Stage 0) exhibited a higher estimated CPU overhead of +2.2% and a memory increase of +5.1 MiB. This efficiency gap is attributed to the architectural differences in monitoring logic: MS-IDS requires the maintenance of a complex process state hash table and continuous I/O counting even in its lowest monitoring stage. Conversely, TRCI employs a lightweight "default-allow, exception-check" strategy, where the NVPM only incurs processing costs when operations trigger the specific paths defined in the *ProtectedList*, thereby minimizing resource contention during routine system activities.

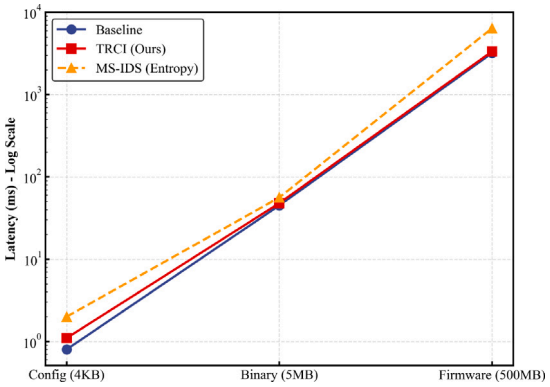


Fig. 14. Launch latency.

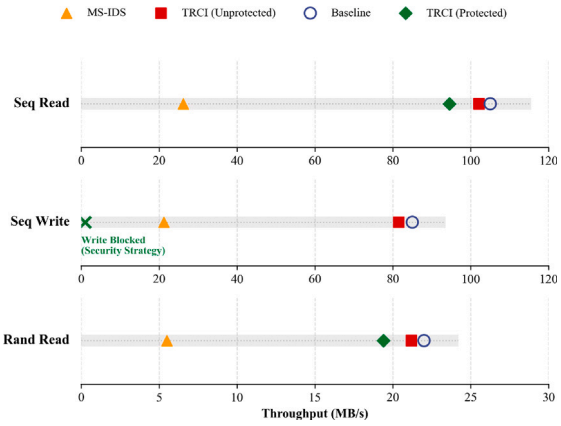


Fig. 15. IO throughput.

6.2.2. Active protection overhead

Next, we assessed the impact of **Lock 1 (NVPM)** on system performance when active protection is engaged. We separated the evaluation into application launch latency and I/O throughput.

Application Launch Latency: We measured the time required to read or execute files of varying sizes: a 4 KB configuration file, a 5 MB binary, and a 500 MB firmware image. As illustrated by the log-scale trends in Fig. 14, the TRCI performance curve closely tracks the Native Baseline, demonstrating excellent scalability across orders of magnitude. For the 500 MB firmware load, the latency increased from 3200 ms to 3350 ms, representing a 4.7% overhead. This overhead stems primarily from the hash calculation performed by the Status Detection Module at the `openat` system call. In contrast, the MS-IDS (Entropy Mode), represented by the dashed line, exhibits a distinct performance gap, incurring a significantly higher overhead (reaching ≈100% for large files) due to the computational intensity of real-time entropy calculation.

I/O Throughput: We further stressed the file system to measure throughput degradation. As illustrated by the performance distribution in Fig. 15, for unprotected paths (general workloads), TRCI demonstrates a throughput loss of less than 5%, attributed solely to path string matching. For protected paths, while write operations are strictly blocked (indicated by the cross marker), the throughput loss for high-frequency reads is controlled within 10%–12%. These results confirm that TRCI's targeted protection strategy offers a more favorable performance-security trade-off for critical edge workloads than full-scope behavior analysis.

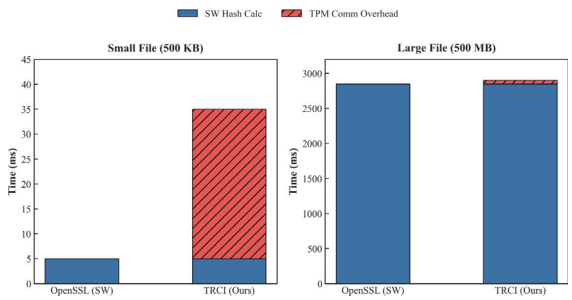


Fig. 16. Hash calculation.

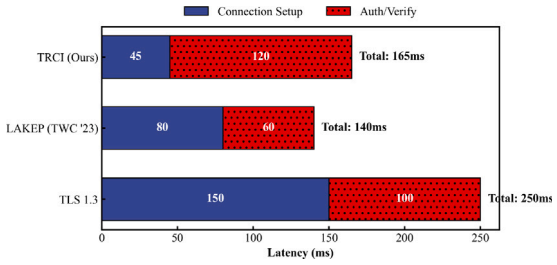


Fig. 17. Communication latency.

6.2.3. Holistic recovery performance

The final set of experiments evaluates the end-to-end efficiency of the recovery process, broken down into three phases: Hash Calculation, Communication Verification, and Execution.

Hash Calculation Latency: Fig. 16 presents the time required to compute SHA-256 digests. For small files (500 KB), TRCI (35 ms) is notably slower than a pure software OpenSSL implementation (4 ms). This latency is dominated by the I2C bus communication overhead (~30 ms) required to extend the TPM PCRs. However, as file size increases to 500 MB, the computation time dominates the I/O overhead. As such, the performance gap narrows significantly (2900 ms for TRCI vs. 2850 ms for OpenSSL), indicating that the TPM overhead becomes negligible for large firmware images.

Communication Verification: To evaluate the efficiency of the control signaling, we compared the TRCI protocol against LAKEP (He et al., 2023) and standard TLS 1.3 (Zhou et al., 2024). Fig. 17 illustrates the latency breakdown, distinguishing between the connection setup phase and the authentication/verification phase.

As observed in the horizontal stacked bar chart, TRCI achieves a faster **Connection Setup** (45 ms, represented by the left segment) compared to LAKEP (80 ms) and TLS 1.3 (150 ms). This advantage is attributed to our simplified 3-way handshake design, which avoids the multi-round negotiation overhead of LAKEP.

However, in the **Auth/Verify Phase** (right segment), TRCI incurs a higher latency of 120 ms compared to LAKEP's 60 ms. This increase is the direct result of performing RSA signature verification within the TPM hardware, whereas LAKEP relies on faster, pure software ECC operations.

Despite the total latency of TRCI (165 ms) being slightly higher than LAKEP (140 ms), it remains significantly more efficient than the standard TLS 1.3 handshake (250 ms). We argue that the additional 25 ms overhead is a justifiable trade-off to obtain hardware-grade protection against side-channel attacks, which is critical for high-stakes control instructions.

Recovery Execution Time: Finally, Fig. 18 illustrates the total time from SEE establishment to system restoration. A distinct characteristic of our hardware-rooted approach is the "Fixed SEE Overhead" of approximately 400 ms, visually distinguished as the red hatched segment

Table 3
Supplementary performance metrics (TRCI scalability).

Metric category	Target evaluation	ARM node	x86 Gateway	Rationale
Steady-State overhead	CPU increment (Δ) Memory increment (Δ)	+0.80% +0.8 MiB	**+0.10%** **+1.2 MiB**	Higher computational redundancy Normal 64-bit structural scaling
I/O Throughput loss	Protected path (Read)	10.0%–12.0%	**<4.5%**	Masked by high L1 Cache hit rates
Control plane latency	Setup + Auth/Verify	165 ms	**149 ms**	Bound by strict HW RSA verification

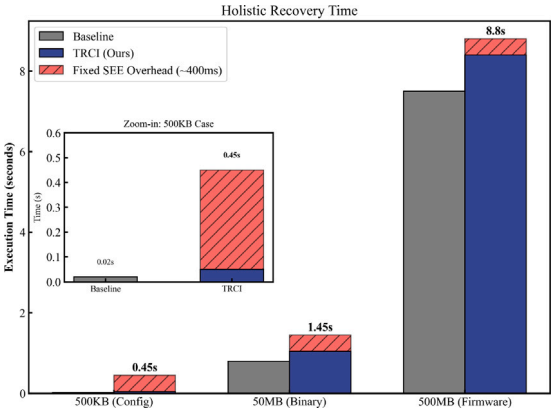


Fig. 18. Recovery revised. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

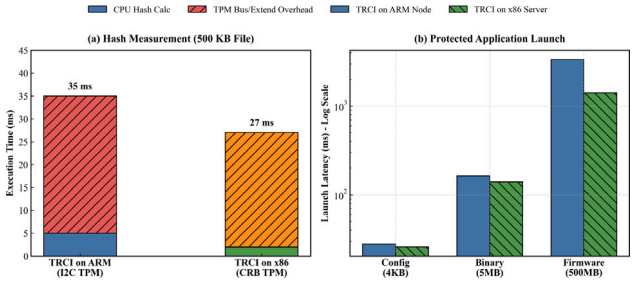


Fig. 19. Elimination of the hardware bottleneck and comparison of launch latency across architectures.

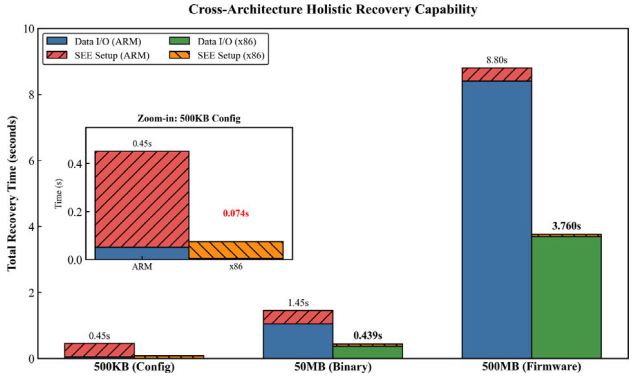


Fig. 20. Efficient cross-platform recovery capability.

stacked atop the I/O time, required for MMU/IOMMU reconfiguration and TLB flushing.

For small configuration files (500 KB), as highlighted in the zoom-in inset, this fixed cost results in a significant relative slowdown factor of 22.5×(0.45 s total vs. 0.02 s baseline). However, for disaster recovery scenarios, an absolute delay of under 0.5 s is operationally acceptable.

Importantly, a "diminishing marginal effect" is observed as the data volume increases. For a 500 MB firmware image, the fixed SEE overhead is diluted by the I/O time (represented by the blue bottom segment), resulting in a total recovery time of 8.80 s compared to the 7.50 s baseline—a slowdown of only 1.17×(17%). This demonstrates that TRCI is highly efficient for its primary use case: the reliable recovery of substantial system assets.

6.3. Architectural scalability on production-grade hardware

The primary evaluation (Sections 6.1 and 6.2) was intentionally conducted on a resource-constrained ARM edge node to amplify overhead sensitivities, providing a clear horizontal comparison among different defense mechanisms under restricted computational and I/O capacities. Subsequently, to isolate and analyze the specific hardware bus bottleneck (I2C) observed in the primary testbed, this section presents a supplementary hardware ablation study. We deployed the framework on a representative x86_64 industrial edge gateway profile (Intel Core i7-7500U, 16 GB RAM, NVMe SSD) running openEuler 20.03 LTS. Crucially, this platform utilizes an integrated Firmware TPM (Intel PTT) that communicates via the high-speed Command Response Buffer (CRB) interface.

As summarized in Table 3, deploying TRCI on a high-performance CPU naturally dilutes the background monitoring costs. The steady-state CPU overhead increment drops to 0.10%, and the I/O throughput loss on protected paths is suppressed below 4.5%, as the fast NVMe I/O and L1/L2 cache efficiencies minimize the context-switching penalty of the system call hooks.

With the baseline metrics confirmed to scale efficiently, we specifically analyzed the latency of the cryptographic anchoring mechanism. As shown in Fig. 19, the measurement architecture separates the bulk hashing task from the hardware attestation. On the x86 platform, the host CPU computes the payload hash rapidly (e.g., 2 ms for a 500 KB file). Subsequently, submitting this 32-byte digest to the TPM via the CRB interface takes approximately 25 ms. While this 27 ms total validation latency represents a notable reduction from the ARM platform (35 ms), we objectively acknowledge that the 35 ms latency observed on the ARM testbed was only partially attributed to the I2C bus overhead of executing this extension, alongside the inherent context-switching delays within the OS-level TPM driver stack.

Finally, Fig. 20 illustrates the cross-architecture holistic recovery capability. Benefiting from rapid page table switching and Non-Maskable Interrupt (NMI) handling on the x86 architecture, the fixed SEE setup penalty is reduced to approximately 70 ms. Coupled with the high-throughput NVMe storage, the framework completes a trusted recovery of a 500 KB critical configuration file in 74 ms, a 50 MB service binary in 439 ms, and a 500 MB firmware image in 3.76 s. These results validate that the Dual-Lock architecture scales efficiently when supported by production-grade hardware components.

7. Discussion and limitations

While TRCI demonstrates robust capabilities in orchestrating hardware primitives to enforce edge resilience, it is imperative to explicitly delineate its security boundaries and operational limitations to align with established cybersecurity taxonomies and avoid overstating its guarantees.

1. Distinction Between Prevention, Detection, and Recovery: It must be clarified that TRCI fundamentally operates as a hardware-anchored resilience and recovery framework, rather than an impenetrable prevention mechanism. Lock 1 (NVPM) proactively mitigates unauthorized storage tampering; however, we humbly acknowledge that this software-level hook cannot fundamentally prevent highly sophisticated, in-memory Ring-0 evasions. In such non-cooperative states, the framework provides reactive detection. While TRCI cannot intrinsically prevent a fully compromised OS from acting maliciously, the hardware TPM guarantees that this compromised state cannot be cryptographically hidden from the remote Resilience Authority. TRCI's core contribution is its guarantee of deterministic trusted recovery, which shifts the security focus from absolute prevention to ensuring eventual trust restoration.

2. Real-Time Operational Boundaries: The empirical evaluation demonstrates that TRCI achieves an end-to-end trusted recovery latency of 74 ms for critical configuration files on production-grade x86 hardware. This latency is highly suitable for delay-tolerant edge workloads—such as asynchronous operational telemetry, 5G MEC computational offloading, and standard IoT gateways (3GPP, 2020). In these environments, system correctness does not strictly depend on sub-second timing guarantees, and temporary execution delays in the magnitude of tens of milliseconds are readily absorbed by application-layer buffers without triggering service outages.

Conversely, the instantiation of the Secure Execution Environment requires masking hardware interrupts to effectively freeze the compromised OS state. This architectural trait makes TRCI fundamentally unsuited for strictly time-triggered, *hard real-time* systems, such as autonomous vehicle drive-by-wire mechanisms or industrial robotics (Kopetz, 2011). In such safety-critical domains, control loops operate under extremely tight temporal bounds. Imposing an unpredictable interrupt masking penalty would completely disrupt the predictability of the real-time scheduler, introducing unacceptable deterministic jitter that could translate directly into catastrophic physical consequences (Gomes et al., 2018).

3. Physical and Infrastructure Threat Exclusions: Consistent with typical edge execution environments, our threat model assumes the physical integrity of the device. TRCI does not provide guarantees against physical hardware tampering, such as direct EEPROM flashing, physical I2C/CRB bus probing, or side-channel key extraction. Furthermore, TRCI cannot mitigate upstream network infrastructure failures or massive external Denial-of-Service (DoS) attacks targeting the control plane. In the event that communication with the RA is persistently severed, the hardware watchdog prioritizes system integrity over availability, defaulting to a localized secure reset, but it cannot autonomously restore distributed network connectivity.

8. Conclusions

As edge infrastructure becomes increasingly critical yet vulnerable, bridging the gap between static hardware trust and dynamic runtime resilience is paramount for surviving advanced persistent threats. Existing solutions often struggle to maintain control when the host operating system is compromised. To address this fundamental challenge, this paper proposed TRCI (Trusted Resilience for Critical Infrastructure), a hardware-anchored framework tailored for the resource-constrained cloud–edge continuum.

TRCI leverages a Dual-Lock mechanism that achieves a robust cross-layer integration of established hardware trust primitives and software-defined policies. By implementing a Resilient Recovery mechanism, our solution effectively secures the control plane, guaranteeing the atomic restoration of a trusted state even when the device enters a "Non-Cooperative State" under adversarial control.

We validated the framework through full-scale prototypes across both ARM-based edge nodes and x86-based industrial platforms. The evaluation confirms that TRCI effectively detects and mitigates both data plane integrity violations and control plane subversion attempts. These security guarantees are achieved with manageable performance overhead, proving that TRCI provides a scalable and operationally efficient defense for critical edge workloads.

While TRCI currently establishes a robust resilience foundation for individual edge nodes, the evolution of edge computing is shifting toward decentralized, large-scale coordination. Therefore, our future work will focus on extending TRCI to support **trusted collaboration in heterogeneous device swarms**.

Specifically, we plan to investigate decentralized remote attestation protocols to establish a collective chain of trust among diverse edge nodes. The goal is to synchronize resilience capabilities across the entire edge infrastructure without relying on a centralized verifier or bottleneck. By enabling peer-to-peer trust verification and collaborative recovery, we aim to enhance the collective survivability of distributed edge networks against large-scale, coordinated attacks.

CRediT authorship contribution statement

Xinzhu Jiang: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Conceptualization. **Bo Zhao:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Peiwen Chen:** Writing – original draft, Data curation. **Chunyu Yang:** Resources, Methodology, Data curation. **Juncheng Tong:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China [grant number 62572356]. The authors would like to thank the anonymous reviewers and the handling editor for their insightful comments and constructive suggestions, which significantly contributed to improving the quality of this manuscript. We also thank our colleagues for their valuable discussions during the course of this research.

Data availability

Data will be made available on request.

References

3GPP, 2020. Service Requirements for Cyber-Physical Control Applications in Vertical Domains. Technical Specification TS 22.104, 3rd Generation Partnership Project (3GPP).

Aaraj, N., Raghunathan, A., Jha, N.K., 2009. Analysis and design of a hardware/software trusted platform module for embedded systems. ACM Trans. Embed. Comput. Syst. (TECS) 8 (1), 1–31.

Al-Otaiby, N.F., Hammoudeh, M., Hassine, J., Bashir, A.K., 2026. Root of trust: Survey, taxonomy, and open challenges. Telecommun. Syst. 89 (1), 39.

Alhidaifi, S.M., Asghar, M.R., Ansari, I.S., 2024. A survey on cyber resilience: Key strategies, research challenges, and future directions. ACM Comput. Surv. 56 (8), 1–48.

Alobaywi, B., Almutairi, M.G., Sheldon, F.T., 2026. Performance trade-offs in multi-tenant IoT–cloud security: A systematic review of emerging technologies. IoT 7 (1), 21.

Arthur, W., Challener, D., 2015. A Practical Guide to TPM 2.0: Using the Trusted Platform Module in the New Age of Security. A Press.

Bartock, M., Franklin, J., Martin, J.A., Roback, M., 2018. Platform Firmware Resiliency Guidelines. NIST Special Publication (SP) 800–193, National Institute of Standards and Technology (NIST), Gaithersburg, MD, http://dx.doi.org/10.6028/NIST.SP.800-193, URL: https://csrc.nist.gov/pubs/sp/800/193/final.

Chenet, C.P., Savino, A., Di Carlo, S., 2024. A survey on hardware-based malware detection approaches. IEEE Access 12, 54115–54128.

China National Vulnerability Database (CNVD), 2026. CNVD-2026-10807: Tenda G0-5g-PoE router denial of service vulnerability. https://www.cnvd.org.cn/flaw/show/CNVD-2026-10807.

Christine, D.I., Thinyane, M., 2022. Socio-technical cyber resilience: A systematic review of cyber resilience management frameworks. In: Digital Transformation for Sustainability: ICT-Supported Environmental Socio-Economic Development. Springer, pp. 573–597.

DaSilva, P.R., Fortier, P.J., 2019. Hardware based detection, recovery, and tamper evident concept to protect from control flow violations in embedded processing. In: 2019 IEEE International Symposium on Technologies for Homeland Security. HST, IEEE, pp. 1–6.

Dharsee, K., 2023. Critical Hardware Towards Software Security Enforcement. University of Rochester.

Dharsee, K., Criswell, J., 2023. Jinn: Hijacking safe programs with trojans. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 6965–6982.

Efe, A., Abacı, İ.N., 2022. Comparison of the host based intrusion detection systems and network based intrusion detection systems.

Eisoldt, J., Galanou, A., Ruzhanskiy, A., Küchenmeister, N., Baburkin, Y., Dai, T., Gudymenko, I., Köpsell, S., Kapitza, R., 2025. SoK: A cloudy view on trust relationships of CVMs–How confidential virtual machines are falling short in public cloud. arXiv preprint arXiv:2503.08256.

Foreman, J.C., 2018. A survey of cyber security countermeasures using hardware performance counters. arXiv preprint arXiv:1807.10868.

Garb, K., Obermaier, J., Ferres, E., Künig, M., 2021. Fortress: FORtified tamper-resistant envelope with embedded security sensor. In: 2021 18th International Conference on Privacy, Security and Trust. PST, IEEE, pp. 1–12.

Geetanshi, Manocha, H., Babbar, H., Mangla, C., 2025. Securing the internet of things: Cybersecurity challenges, strategies, and future directions in the era of 5G and edge computing. In: Current and Future Cellular Systems: Technologies, Applications, and Challenges. Wiley Online Library, pp. 89–106.

Gomes, C., Thule, C., Broman, D., Larsen, P.G., Vangheluwe, H., 2018. Co-simulation: A survey. ACM Comput. Surv. 51 (3), 1–33.

Gupta, S., 2023. Retracted: An edge-computing based industrial gateway for industry 4.0 using ARM TrustZone technology.

Gyányi, S., 2024. Security implications of computer botnets.

He, D., Cai, Y., Zhu, S., Zhao, Z., Chan, S., Guizani, M., 2023. A lightweight authentication and key exchange protocol with anonymity for IoT. IEEE Trans. Wirel. Commun. 22 (11), 7862–7872.

Huang, H., Wang, J., 2025. Systematic literature review on security mechanisms for IoT device firmware update. Available at SSRN 5359954.

Huang, H., Zhang, F., Yan, S., Wei, T., He, Z., 2024. SoK: A comparison study of arm TrustZone and CCA. In: 2024 International Symposium on Secure and Private Execution Environment Design. SEED, IEEE, pp. 107–118.

Hubbard, G.A., 2023. State-level cyber resilience: A conceptual framework. Appl. Cybersecur. Internet Gov. 2 (1), 1–14.

Jain, B., Baig, M.B., Zhang, D., Porter, D.E., Sion, R., 2014. Sok: Introspections on trust and the semantic gap. In: 2014 IEEE Symposium on Security and Privacy. IEEE, pp. 605–620.

Kopetz, H., 2011. Real-Time Systems: Design Principles for Distributed Embedded Applications. Springer Science & Business Media.

Kwon, H.-Y., Kim, T., Lee, M.-K., 2022. Advanced intrusion detection combining signature-based and behavior-based detection methods. Electronics 11 (6), 867.

Latif, S., Djenouri, D., Idrees, Z., Ahmad, J., Zou, Z., et al., 2025. Hardware security modules for secure communications in the industrial internet of things. IEEE Commun. Surv. Tutor..

Litchfield, A., Du, W., 2023. Xfilter: An extension of the integrity measurement architecture based on fine-grained policies. Appl. Sci. 13 (10), 6046.

Liu, H., Jiang, R., 2023. A causal graph-based approach for apt predictive analytics. Electronics 12 (8), 1849.

Mahavaishnavi, V., Saminathan, R., Prithviraj, R., 2025. Secure container orchestration: A framework for detecting and mitigating orchestrator-level vulnerabilities. Multimedia Tools Appl. 84 (17), 18351–18371.

Marchand, A., Imine, Y., Ouarnoughi, H., Tarridec, T., Gallais, A., 2023. Firmware integrity protection: A survey. IEEE Access 11, 77952–77979.

Marchetti, E., Costa, A., Nikghadam-Hojjati, S., Barata, J., Calabrò, A., 2026. Toward cybersecure-by-design manufacturing systems: A taxonomy of hardware-based approaches for software cybersecurity violation detection. In: Barata, J., Madani, K., Panetto, H. (Eds.), Innovative Intelligent Industrial Production and Logistics. Springer Nature Switzerland, Cham, pp. 372–392.

Martins, I., Resende, J.S., Sousa, P.R., Silva, S., Antunes, L., Gama, J., 2022. Host-based IDS: A review and open issues of an anomaly detection system in IoT. Future Gener. Comput. Syst. 133, 95–113.

Muin, Y., et al., 2022. MikroTik router vulnerability testing for network vulnerability evaluation using penetration testing method. Int. J. Comput. Appl. 975, 8887.

Ott, S., Kamhuber, M., Pecholt, J., Wessel, S., 2023. Universal remote attestation for cloud and edge platforms. In: Proceedings of the 18th International Conference on Availability, Reliability and Security. pp. 1–11.

Pandey, V.K., Prakash, S., Singh, S., Yang, T., Rathore, R.S., et al., 2025. An efficient approach for side channel attack in cloud computing. Procedia Comput. Sci. 258, 1404–1413.

Peddoju, S.K., Upadhyay, H., Lagos, L., 2020. File integrity monitoring tools: Issues, challenges, and solutions. Concurr. Comput.: Pr. Exp. 32 (22), e5825.

Pettersen, R., Johansen, H.D., Johansen, D., 2017. Secure edge computing with ARM TrustZone. In: IoTBDS. pp. 102–109.

Rahman, F., Farmani, M., Tehranipoor, M., Jin, Y., 2017. Hardware-assisted cybersecurity for IoT devices. In: 2017 18th International Workshop on Microprocessor and SOC Test and Verification. MTV, IEEE, pp. 51–56.

Regenscheid, A., 2017. Platform Firmware Resiliency Guidelines. Technical Report, National Institute of Standards and Technology.

Ross, R., Pillitteri, V., Graubart, R., Bodeau, D., McQuaid, R., 2021. Developing Cyber-Resilient Systems: A Systems Security Engineering Approach. NIST Special Publication (SP) 800-160 Vol. 2 Rev. 1, National Institute of Standards and Technology (NIST), Gaithersburg, MD, http://dx.doi.org/10.6028/NIST.SP.800-160v2r1, URL: https://csrc.nist.gov/pubs/sp/800/160/v2/r1/final.

Rother, C., Chen, B., 2024. Reversing file access control using disk forensics on low-level flash memory. J. Cybersecur. Priv. 4 (4), 805–822.

Sheikh, A.M., Islam, M.R., Habaebi, M.H., Zabidi, S.A., Bin Najeeb, A.R., Kabbani, A., 2025. A survey on edge computing (EC) security challenges: Classification, threats, and mitigation strategies. Futur. Internet 17 (4), 175.

van Sloun, C., Woeste, V., Wolsing, K., Pennekamp, J., Wehrle, K., 2025. Detecting ransomware despite I/O overhead: A practical multi-staged approach. In: NDSS.

Surve, P.P., Brodt, O., Yampolskiy, M., Elovici, Y., Shabtai, A., 2023. SoK: Security below the OS–A security analysis of UEFI. arXiv preprint arXiv:2311.03809.

Syed, A.A.M., 2024. Disaster recovery and data backup optimization: Exploring next-gen storage and backup strategies in multi-cloud architectures. Int. J. Emerg. Res. Eng. Technol. 5 (3), 32–42.

Ta, P.B., Mafeni, V., Kim, Y., 2025. A design of workflow-based automated failure recovery framework in IoT edge environment. IEEE Access.

Tamimi, A.A., Dawood, R., Sadaqa, L., 2019. Disaster recovery techniques in cloud computing. In: 2019 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology. JEEIT, IEEE, pp. 845–850.

Ul Haq, S., Singh, Y., Sharma, A., Gupta, R., Gupta, D., 2023. A survey on IoT & embedded device firmware security: Architecture, extraction techniques, and vulnerability analysis frameworks. Discov. Internet Things 3 (1), 17.

Upadhyay, R., Paudyal, R., Donnan, L., Wijesekera, D., et al., 2025. TPM-based continuous remote attestation and integrity verification for 5G VNFs on kubernetes. In: 2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications. TPS-ISA, IEEE, pp. 435–444.

Wang, Y., Liu, H., Li, Z., Su, Z., Li, J., 2024. Combating advanced persistent threats: Challenges and solutions. IEEE Netw. 38 (6), 324–333.

Yang, M., Qu, Y., Ranbaduge, T., Thapa, C., Sultan, N.H., Ding, M., Suzuki, H., Ni, W., Abuadbba, S., Smith, D., et al., 2026. From 5g to 6g: A survey on security, privacy, and standardization pathways. ACM Comput. Surv. 58 (8), 1–38.

Zhou, J., Fu, W., Hu, W., Sun, Z., He, T., Zhang, Z., 2024. Challenges and advances in analyzing tls 1.3-encrypted traffic: A comprehensive survey. Electronics 13 (20), 4000.